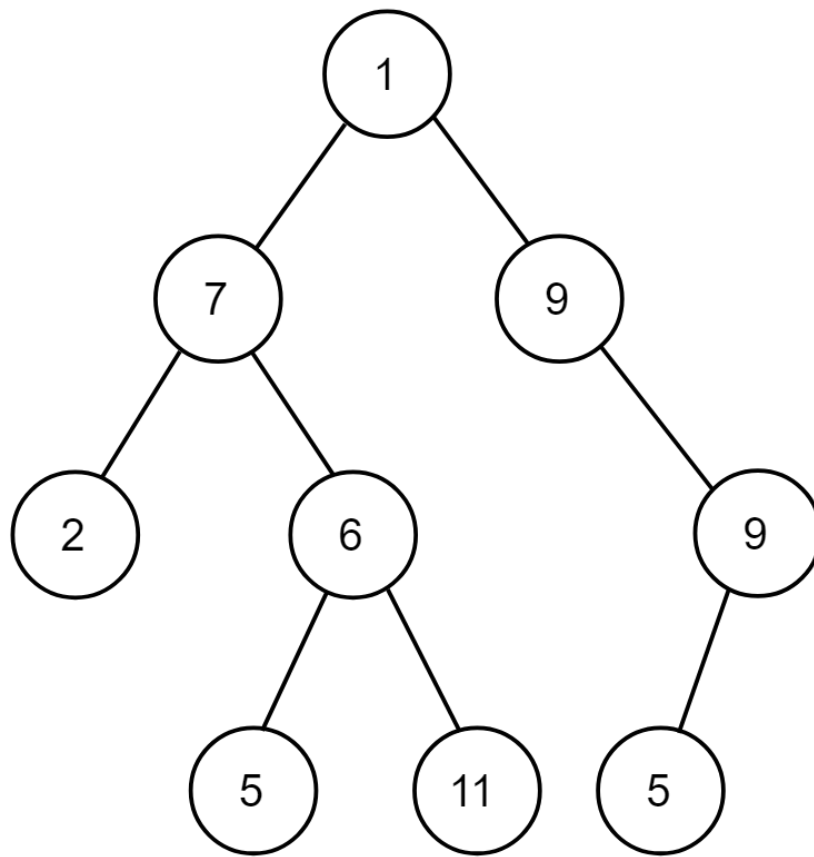
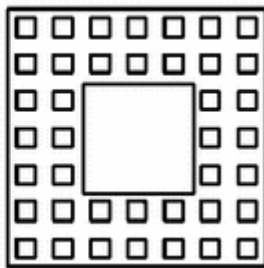
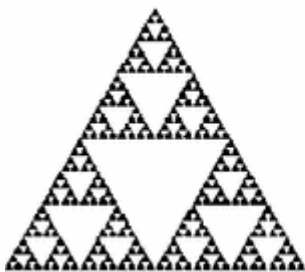




Recursion

Trees and Fractals



schedule

<u>Class #</u>	<u>Date</u>	<u>Topic</u>	<u>Content</u>
1	8-Jan	Intro	Python introduction, grid based geometries
1	15-Jan	Python deep dive	Pytho online resources (W3C)
2	22-Jan	Applications: RhinoScript, Grasshopper	RhinoScript library
3	29-Jan	Object Oriented Programming & Rhino Common	W3C Class documentation, RhinoCommon API
4	5-Feb	Linear Algebra and Transformations	Points, lines, planes, transforms
5	12-Feb	Curves	Curves & Beziers
6	19-Feb	Surfaces	Surfaces & Curvature
7	26-Feb	Breps & meshes	Breps, meshes
8	5-Mar	Spring Break - no class	Tabular data, excel integration
9	12-Mar	Data 1 - Tabular Data	JSON / unstructured data
10	19-Mar	Data 2 - Trees & Lists	
11	26-Mar	Recursion	
12	2-Apr	AI & Vibe programming	VS Code and CoPilot
13	9-Apr	Inputs	Eto input forms
14	16-Apr	TBD	
15	23-Apr	TBD	
16	TBD	Final exam / presentation	

Recursive programming

Recursive programming is a technique where **a function calls itself** to solve a problem. The function continues to call itself with **simpler or smaller inputs** until it reaches a **base case** that stops the recursion.

✓ Key Components of Recursion:

1. Base Case:

- The condition that stops the recursion.
- Without a base case, the function would call itself infinitely, causing a stack overflow.

2. Recursive Case:

- The part of the function that calls itself with modified input.

Basic recursive iteration

recursion 00 - counting.py

```
✓ def iter(aLevel):  
    #prints before iterating in - while going down  
    print(f"going down: {aLevel}")  
  
    ✓ if (aLevel):  
        iter(aLevel - 1)  
  
    #prints after iterating in - while coming up  
    print(f"coming up: {aLevel}")  
  
iter(10)
```

Basic recursive iteration

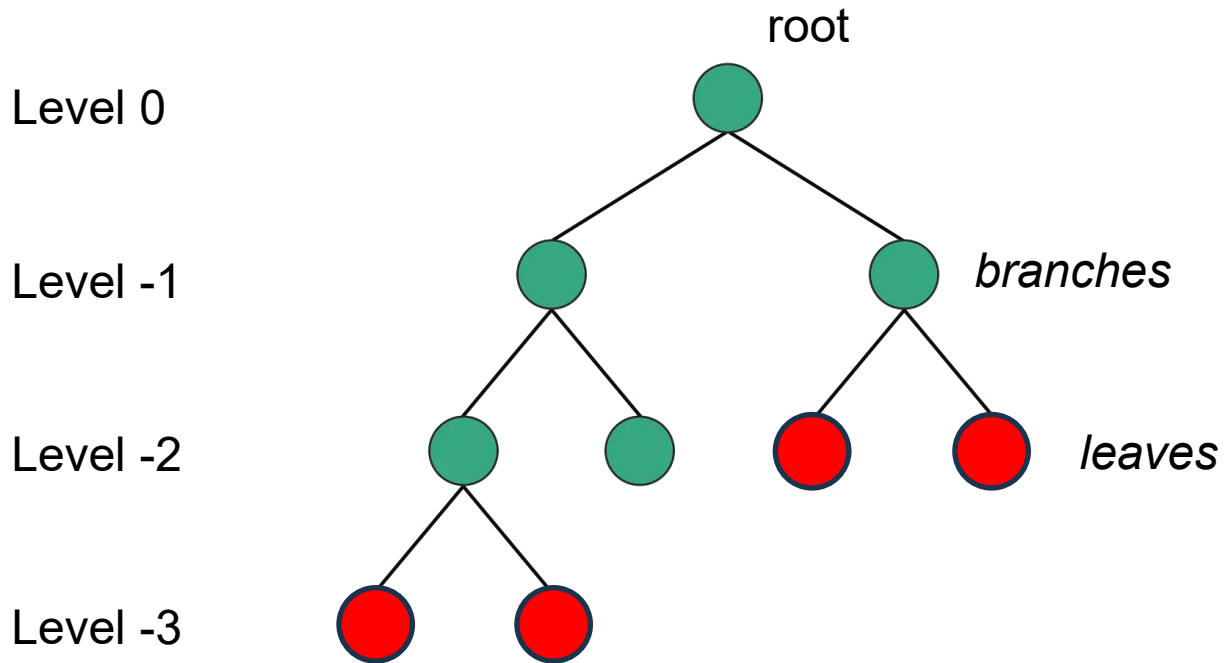
recursion 00 - counting.py

```
✓ def iter(aLevel):  
    #prints before iterating in - while going down  
    print(f"going down: {aLevel}")  
  
    if (aLevel):  
        iter(aLevel - 1)  
  
    #prints after iterating in - while coming up  
    print(f"coming up: {aLevel}")
```

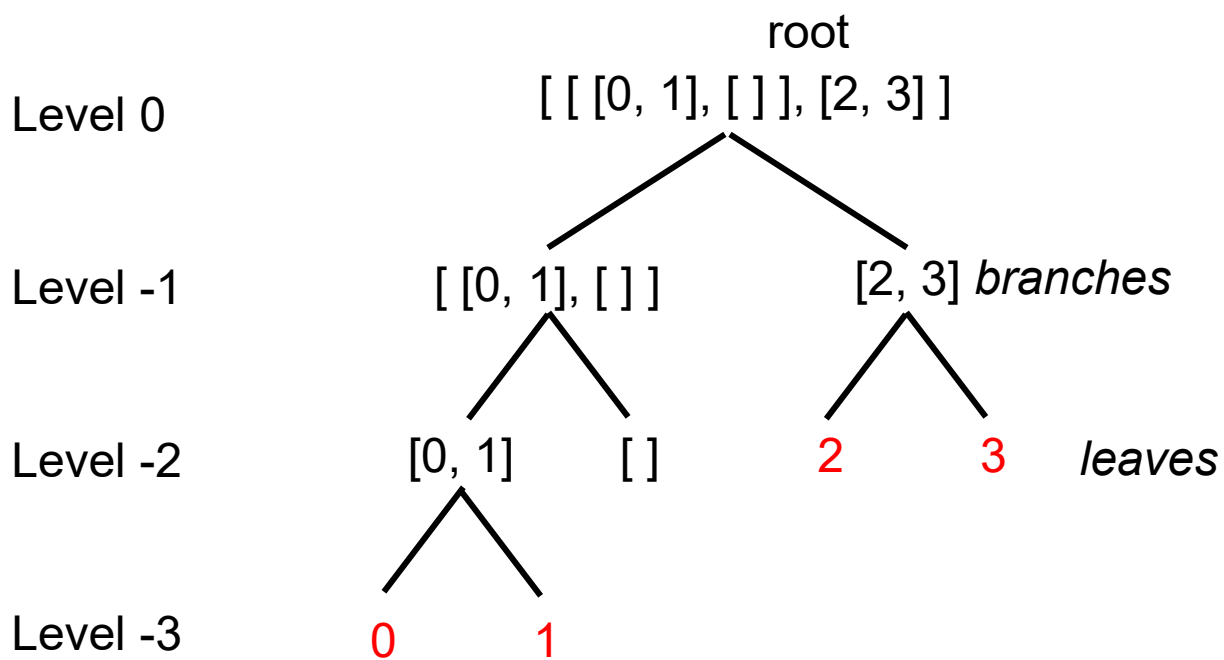
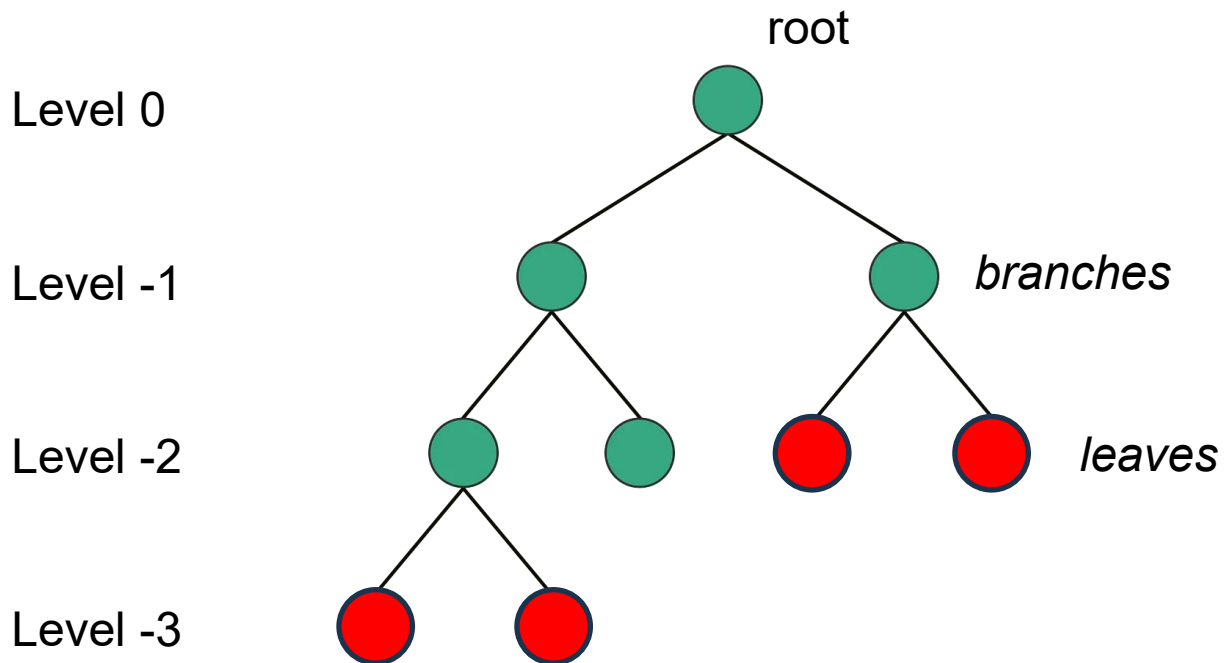
```
iter(10)
```

```
going down: 10  
going down: 9  
going down: 8  
going down: 7  
going down: 6  
going down: 5  
going down: 4  
going down: 3  
going down: 2  
going down: 1  
going down: 0  
coming up: 0  
coming up: 1  
coming up: 2  
coming up: 3  
coming up: 4  
coming up: 5  
coming up: 6  
coming up: 7  
coming up: 8  
coming up: 9  
coming up: 10
```

Iterating through a tree



Iterating through a tree



Creating a binary tree

```
def iter(level):
```

Definition of *iter*

```
    newtree = []
```

```
    if level == 0:
```

termination

```
        global i
```

```
        i += 1
```

```
        ret = [i]
```

```
        return ret
```

```
    newtree.append(iter(level - 1))
```

Recursively calls *iter*

```
    newtree.append(iter(level - 1))
```

```
    return newtree
```

```
i = 0
```

```
a = []
```

```
a.append(iter(levels))
```

Initial call *iter*

```
print(a)
```

Creating a binary tree

```
def iter(level):
```

Definition of *iter*

```
    newtree = []
```

```
    if level == 0:
```

termination

```
        global i
```

```
        i += 1
```

```
        ret = [i]
```

```
        return ret
```

```
    newtree.append(iter(level - 1))
```

Recursively calls *iter*

```
    newtree.append(iter(level - 1))
```

```
    return newtree
```

```
i = 0
```

```
a = []
```

```
a.append(iter(levels))
```

Initial call *iter*

```
print(a)
```

Reading a binary tree

```
✓ def iter(tree, ret):
```

Definition of *iter*

```
✓     if isinstance(tree, list):
```

```
✓         for i in tree:
```

```
             iter(i, ret)
```

Recursively calls *iter*

```
✓     else:
```

```
         ret.append(tree)
```

termination

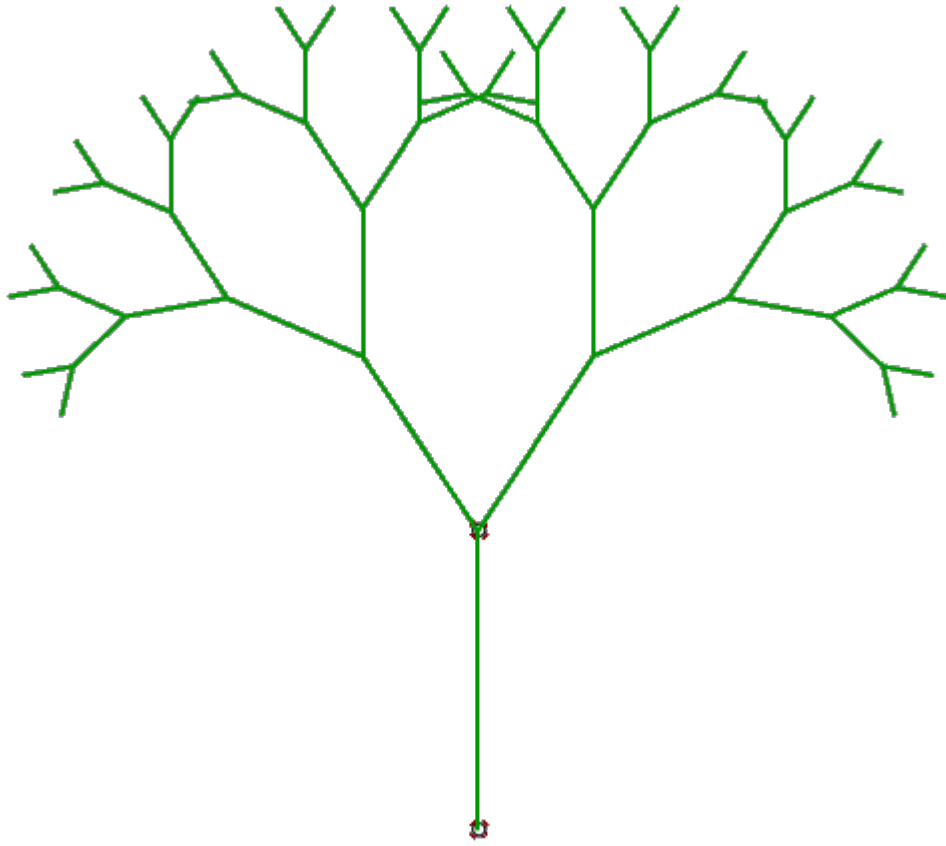
```
ret = []
```

```
iter(t, ret)
```

Initial call *iter*

```
print(ret)
```

Drawing a tree



Drawing a tree

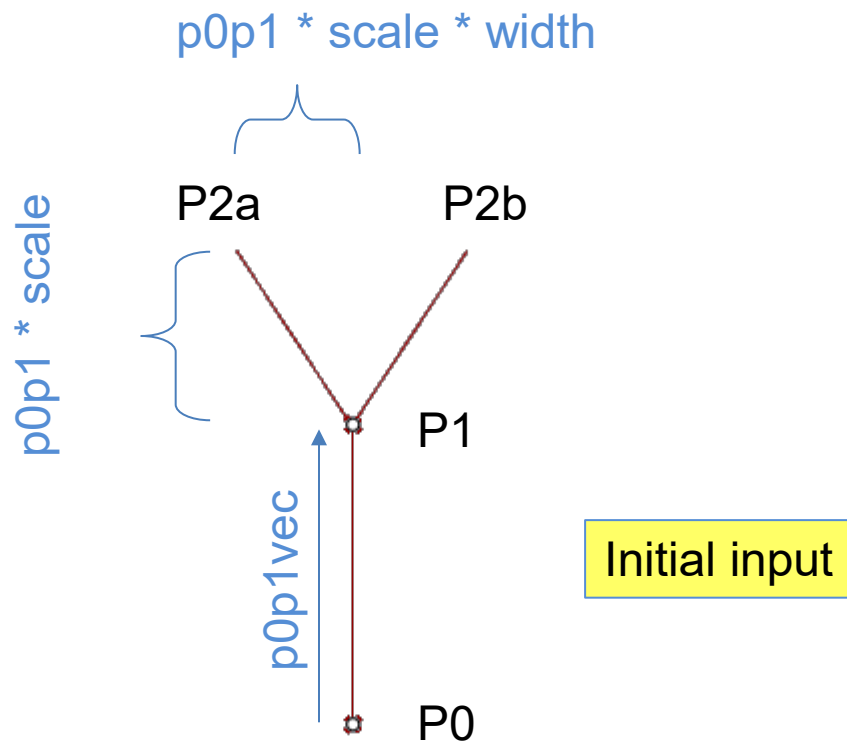
Start – Level 0



Initial input

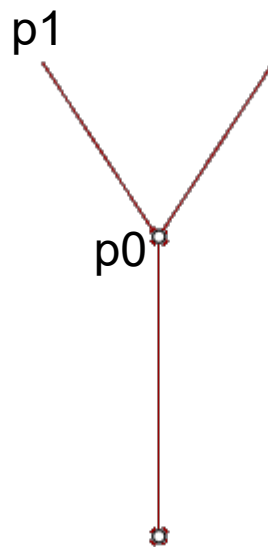
Drawing a tree

Start – Level 0



Drawing a tree

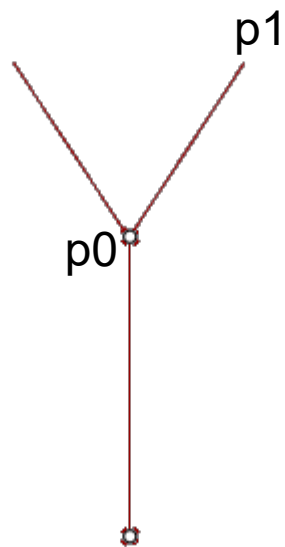
Start – Level 1 (a)



Level 1 (a) input

Drawing a tree

Start – Level 1 (b)



Level 1 (b) input

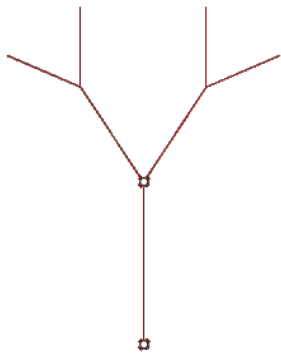
Drawing a tree



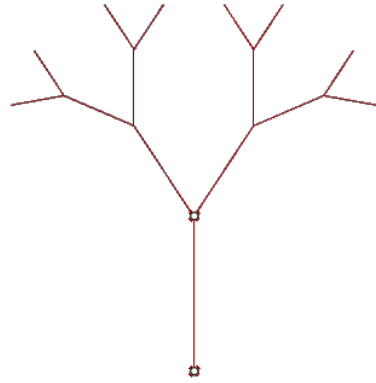
Depth 0



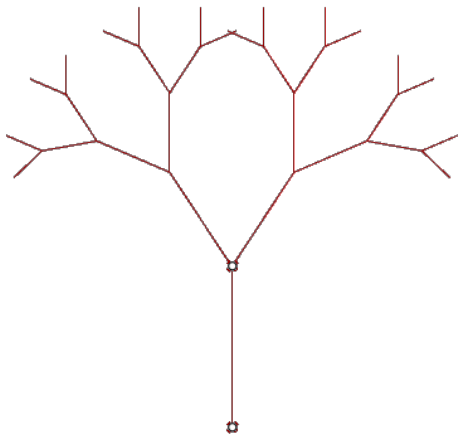
Depth 1



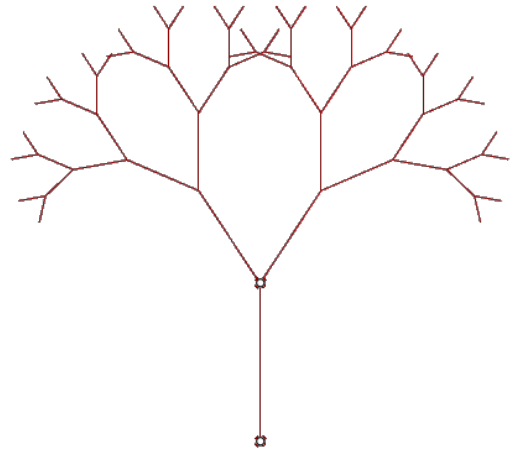
Depth 2



Depth 3



Depth 4



Depth 5

Drawing a tree

02 - Tree Recursion.gh

```
import rhinoscriptsyntax as rs
import Rhino
import math

def iter(p0, p1, level):
    a.append(rs.AddLine(p0, p1))

    p03d = Rhino.Geometry.Point3d(p0)
    p13d = Rhino.Geometry.Point3d(p1)

    p0p1vec = p13d - p03d # Vector p0->p1
    p0p1VecLen = p0p1vec.Length #calculate distance
    # find the unit vector perpendicular to p0p1
    vcross = Rhino.Geometry.Vector3d.CrossProduct(p0p1vec, Rhino.Geometry.Vector3d.ZAxis)
    vcross.Unitize()

    p2a = p1 + p0p1vec * scale + vcross * p0p1VecLen * scale * width
    p2b = p1 + p0p1vec * scale - vcross * p0p1VecLen * scale * width

    if (level == 0):
        return

    iter(p1, p2a, level - 1)
    iter(p1, p2b, level - 1)

a = []

P0 = rs.coerce3dpoint(p0)
P1 = rs.coerce3dpoint(p1)
iter(Rhino.Geometry.Point3d(P0), Rhino.Geometry.Point3d(P1), levels)
```

Drawing a tree

02 - Tree Recursion.gh

```
import rhinoscriptsyntax as rs
import Rhino
import math

def iter(p0, p1):
    a.append(rs.AddLine(p0, p1))

    p03d = Rhino.Geometry.Point3d(p0)
    p13d = Rhino.Geometry.Point3d(p1)

    p0p1vec = p13d - p03d # Vector p0->p1
    p0p1VecLen = p0p1vec.Length #calculate distance
    # find the unit vector perpendicular to p0p1
    vcross = Rhino.Geometry.Vector3d.CrossProduct(p0p1vec, Rhino.Geometry.Vector3d.ZAxis)
    vcross.Unitize()

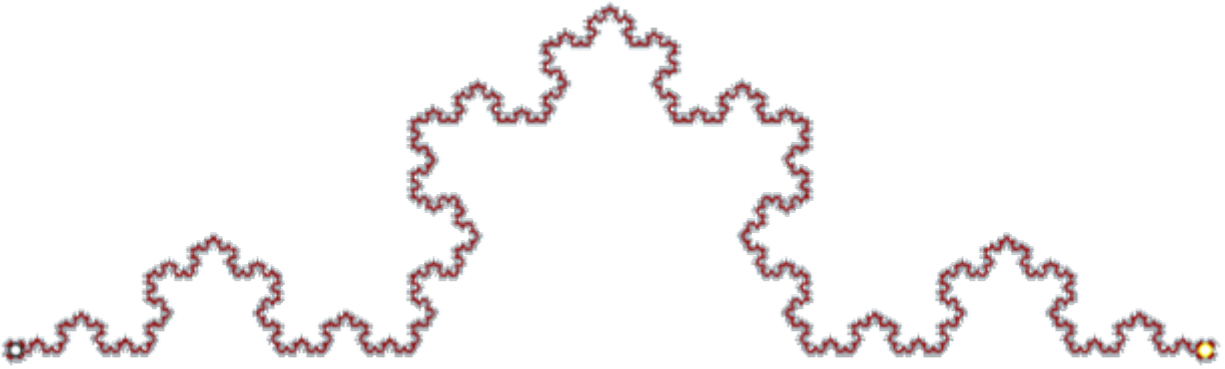
    p2a = p1 + p0p1vec * scale + vcross * p0p1VecLen * scale * width
    p2b = p1 + p0p1vec * scale - vcross * p0p1VecLen * scale * width
    ... # a different approach - terminates based on some condition not level
    ... if (p0p1VecLen < length):
        return

    iter(p1, p2a)
    iter(p1, p2b)

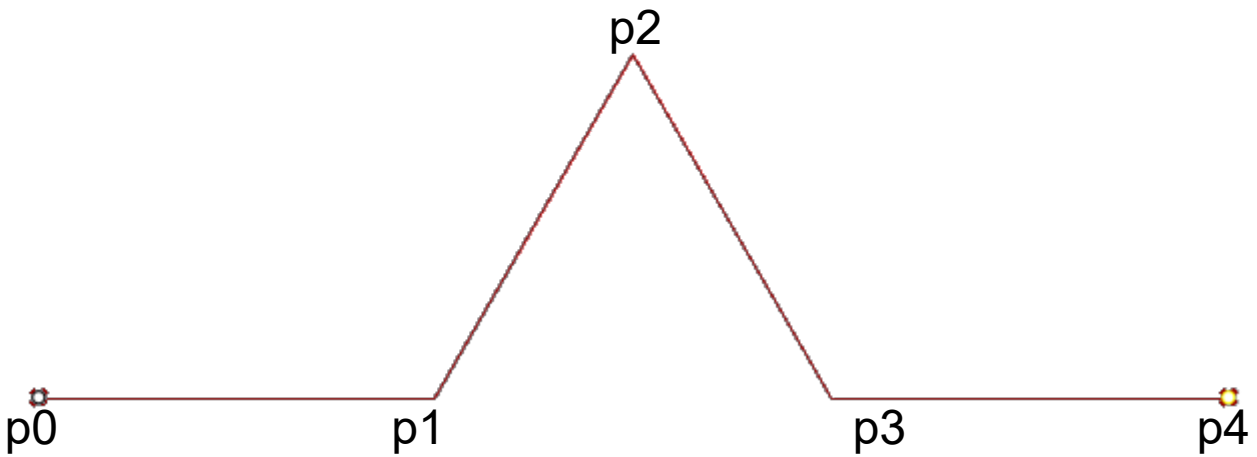
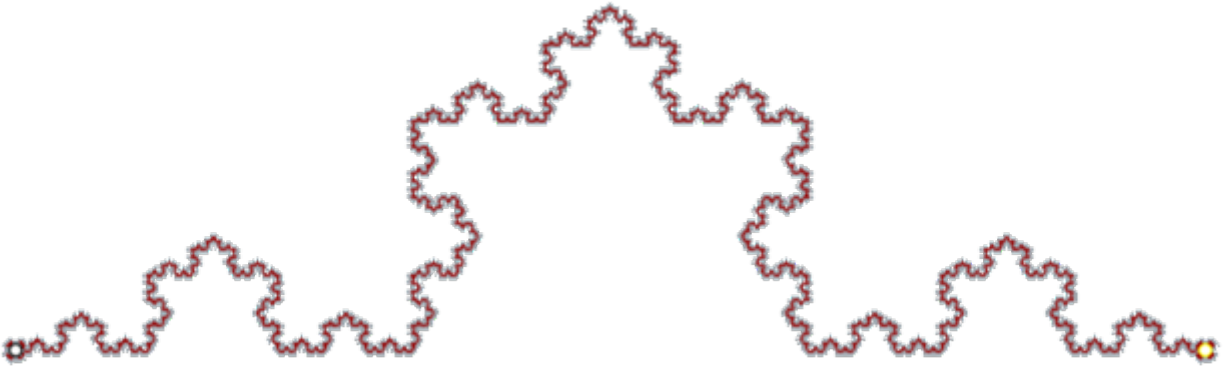
a = []

P0 = rs.coerce3dpoint(p0)
P1 = rs.coerce3dpoint(p1)
iter(Rhino.Geometry.Point3d(P0), Rhino.Geometry.Point3d(P1))
```

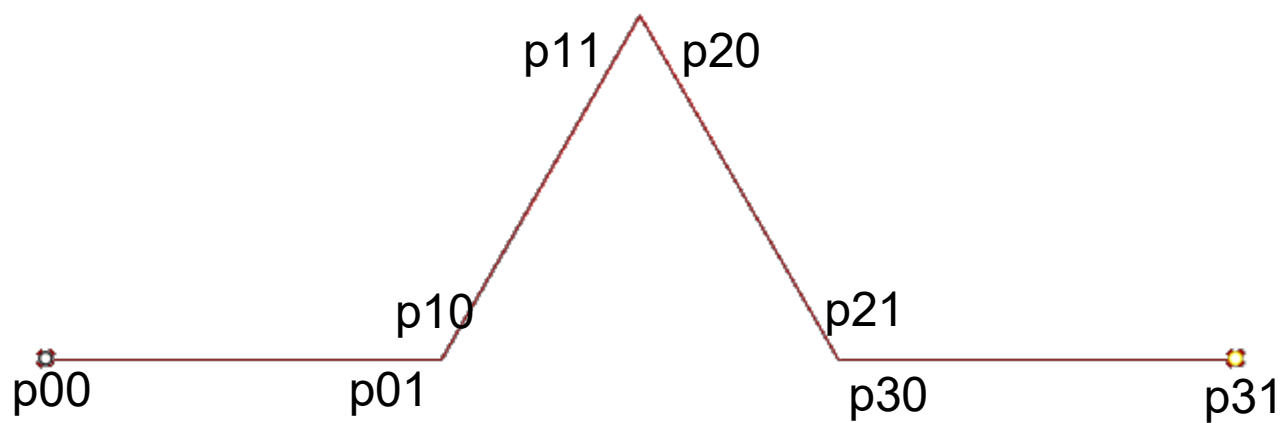
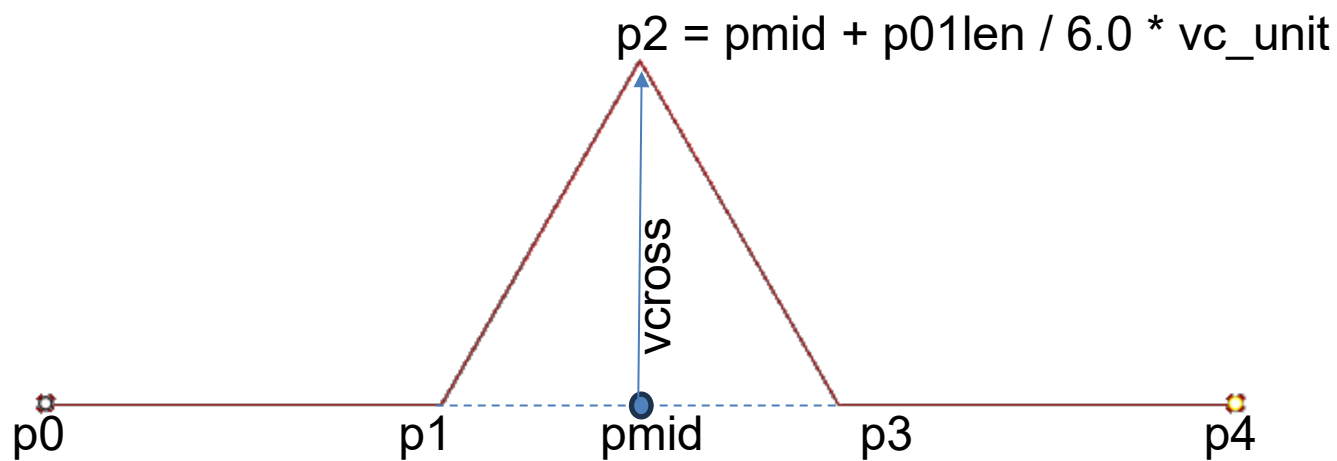
Koch Curve



Koch Curve



Koch Curve



Koch Curve

03 - Koch Curve.gh

```
import rhinoscriptsyntax as rs
import Rhino
import math

def iter(p0, p4, level):
    p1 = (p0 * 2.0 + p4) / 3.0
    p3 = (p0 + p4 * 2.0) / 3.0
    pmid = (p0 + p4) / 2.0

    p0p4vec = p4 - p0
    p0p4len = p0p4vec.Length

    vcross = Rhino.Geometry.Vector3d.CrossProduct(p0p4vec, Rhino.Geometry.Vector3d.ZAxis)
    vcross.Unitize()

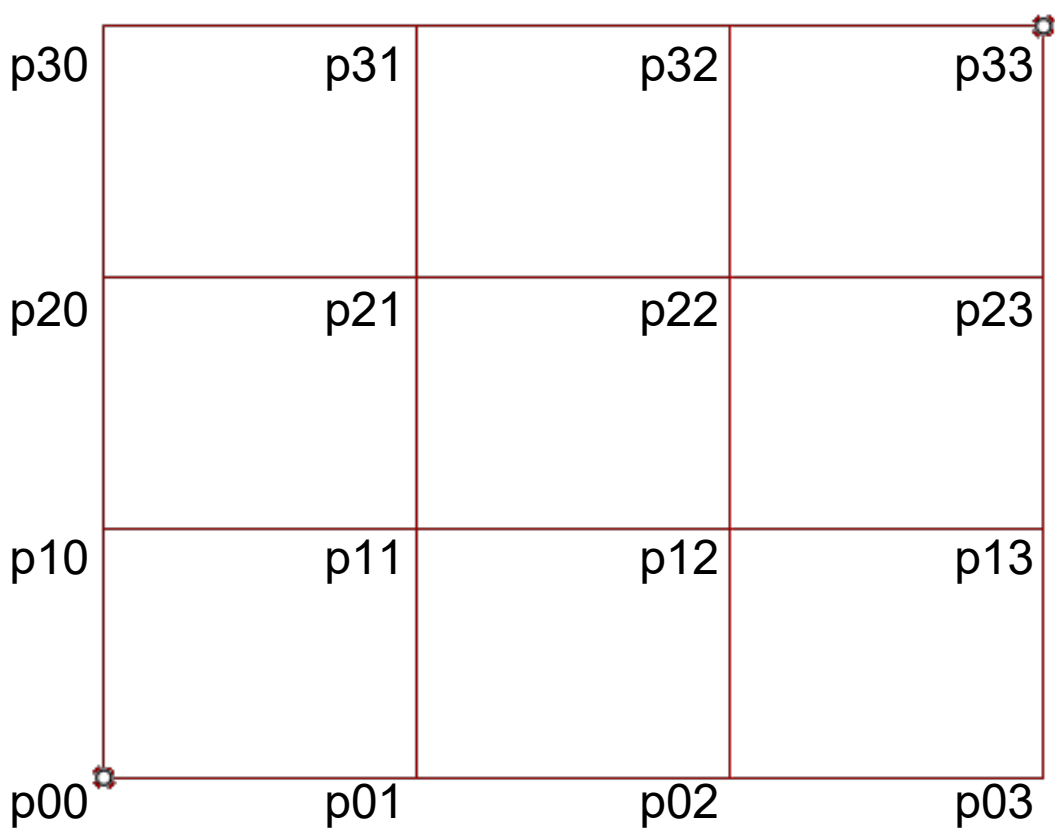
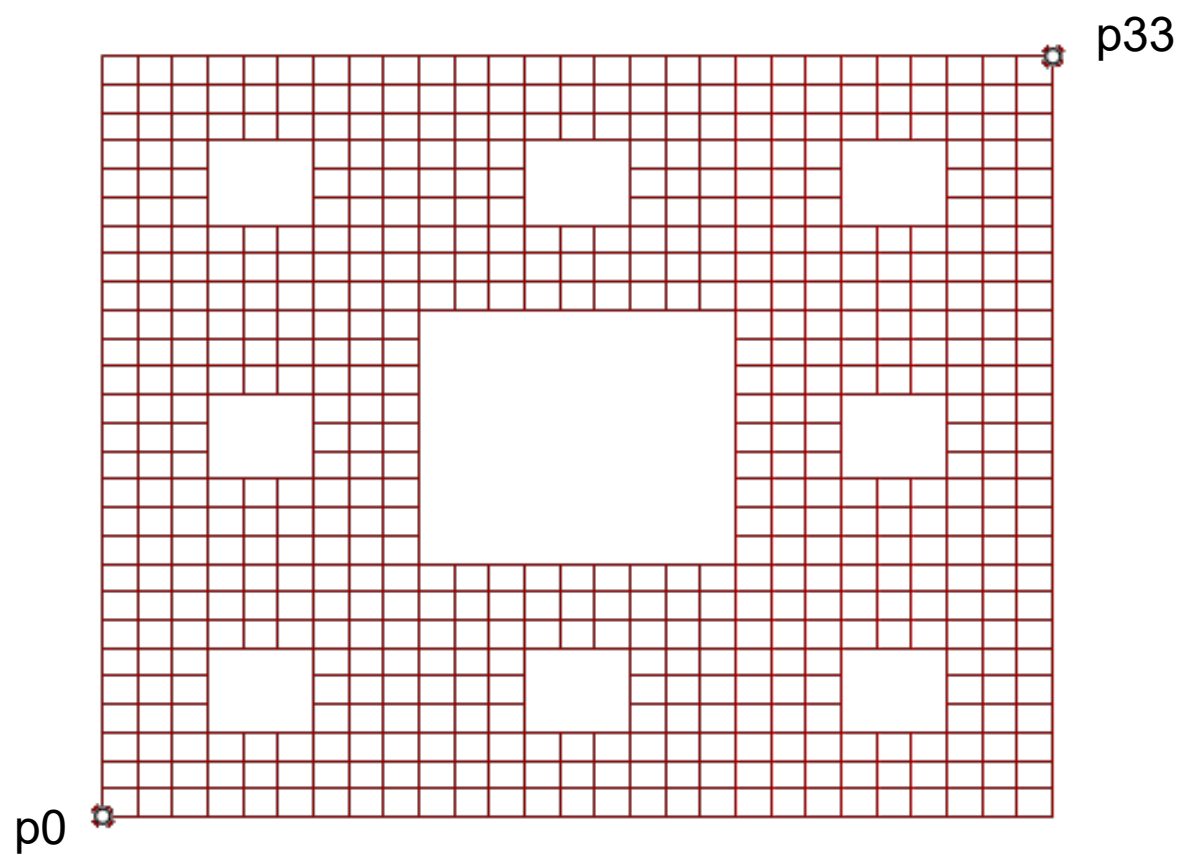
    p2 = pmid - vcross * p0p4len / 6.0 * math.sqrt(3)

    if (level == 0):
        a.append(rs.AddLine(p0, p4))
        return

    iter(p0, p1, level - 1)
    iter(p1, p2, level - 1)
    iter(p2, p3, level - 1)
    iter(p3, p4, level - 1)

a = []
iter(rs.coerce3dpoint(p0), rs.coerce3dpoint(p1), levels)
```

Sierpinski gasket



```
import rhinoscriptsyntax as rs
```

```
import Rhino
```

```
import math
```

```
def iter(p00, p33, level):
```

```
    p00p33vec = p33 - p00
```

```
    xvec = Rhino.Geometry.Vector3d(p00p33vec[0] / 3.0, 0, 0)
```

```
    yvec = Rhino.Geometry.Vector3d(0, p00p33vec[1] / 3.0, 0)
```

```
    #p00 from input
```

```
    p01 = p00 + xvec
```

```
    p02 = p01 + xvec
```

```
    p03 = p02 + xvec
```

```
    ...
```

```
    p10 = p00 + yvec
```

```
    p11 = p10 + xvec
```

```
    p30 = p20 + yvec
```

```
    p31 = p30 + xvec
```

```
    p32 = p31 + xvec
```

```
    #p33 from input
```

```
    if (level == 0):
```

```
        curvepts = []
```

```
        curvepts.append(p00)
```

```
        curvepts.append(p03)
```

```
        curvepts.append(p33)
```

```
        curvepts.append(p30)
```

```
        curvepts.append(p00)
```

```
        a.append(rs.AddPolyline(curvepts))
```

```
        return
```

```
    iter(p00, p11, level - 1)
```

```
    iter(p01, p12, level - 1)
```

```
    ...
```

```
    iter(p21, p32, level - 1)
```

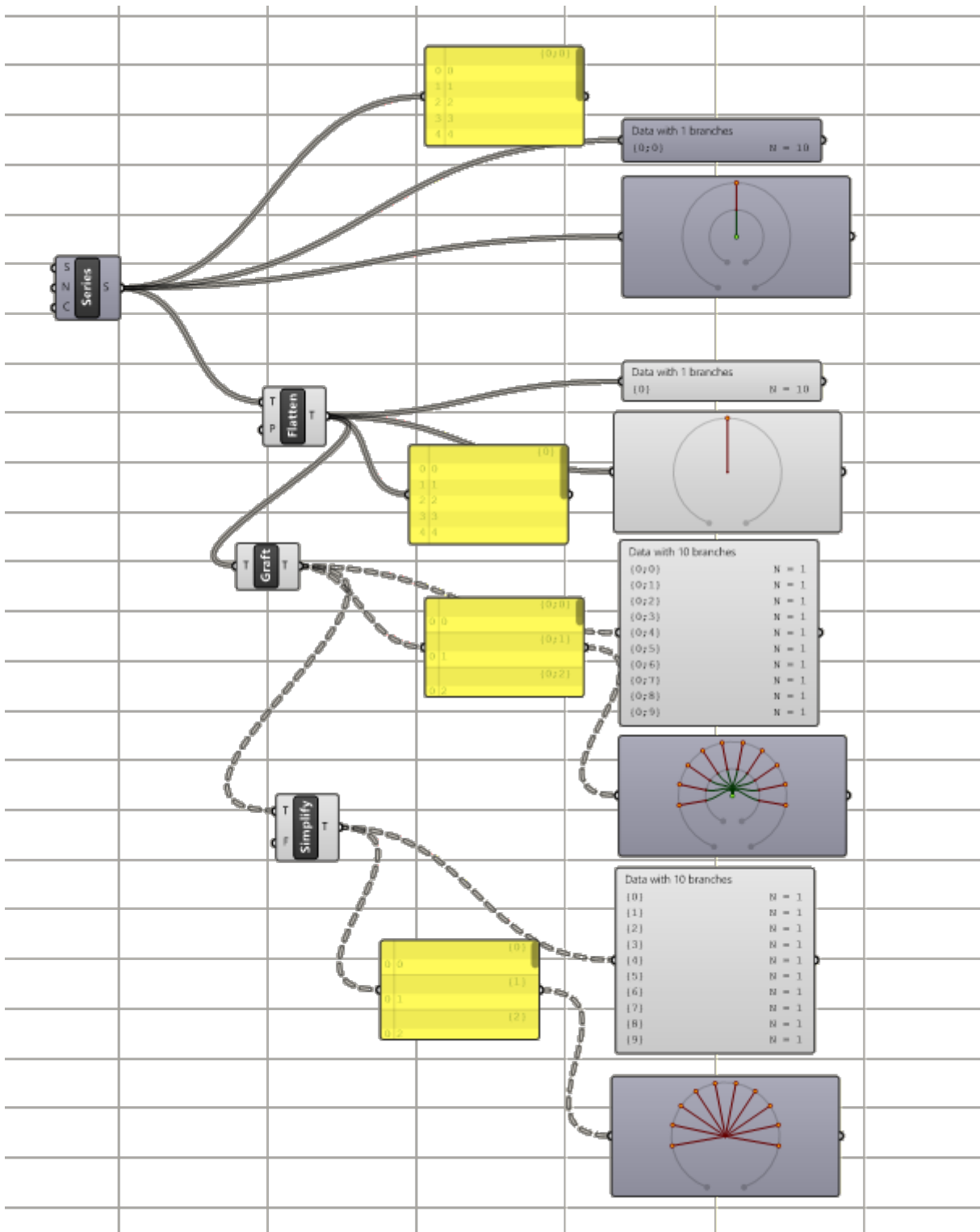
```
    iter(p22, p33, level - 1)
```

```
a = []
```

```
iter(rs.coerce3dpoint(p0), rs.coerce3dpoint(p1), levels)
```

Lists and Data Trees

05. tree manipulations.gh



Lists and Data Trees

06 Tree Conversion.gh

```
# Create a Grasshopper DataTree
tree = DataTree[object]()

def add_to_tree(tree, data, path):
    """
    Recursively traverses a nested list and adds items

    Parameters:
    - tree: Grasshopper DataTree to populate
    - data: Current level of the nested list
    - path: GH_Path representing the current branch
    """
    if isinstance(data, list):
        for i, item in enumerate(data):
            # Extend the path with the current index
            new_path = path.AppendElement(i)
            add_to_tree(tree, item, new_path)
    else:
        # Add the item to the tree
        tree.Add(data, path)

# Initialize with an empty path
initial_path = GH_Path()

# Add the nested list data to the tree
add_to_tree(tree, inputlist, initial_path)

# Output the tree to Grasshopper
a = tree
```